codex-plugin-cc

A Codex crank engine for Claude Code — review · adversarial · execute

Design Specification | v1 | Swimlane 1 of the multi-engine plan | Author: Luan Moreno | Status: DRAFT for build

This document is a **self-contained build brief**. A fresh agent session (Claude, Codex, or Kimi) should be able to implement **codex-plugin-cc** end-to-end from this file alone, with no prior context. It specifies the goal, the proven blueprint it ports, the exact CLI primitives it binds to, the file manifest, every command contract, the shared crank spine, exit codes, the eval gate, and a phased build plan with acceptance checks.

> **One-sentence scope**
>
> Take OpenAI's review-only Codex plugin and add the missing capability — a **task-cranking worker** (`/codex:crank`, `crank-next`, `crank-batch`) — by porting the Kimi plugin's orchestration spine onto the `codex exec` headless CLI, gated by the same Task-Spec eval so "eval passes = done" holds regardless of engine.

1 Why this exists

Claude Code can already *delegate* to other coding agents through plugins. Two exist today in this environment:

- **kimi-plugin-cc** (yours) — a full engine: review, adversarial *challenge*, and a **crank** that executes Task-Spec files (`tasks/T-*.md`) with an eval gate, wave scheduler, and background jobs.
- **codex** (OpenAI official) — deliberately **review + rescue only**: `/codex:review`, `/codex:adversarial-review`, `/codex:rescue`. It has **no task-file runner**, no backlog scheduler, no eval gate.

The gap is exact: **Codex can review and rescue, but it cannot crank a backlog.** This plugin closes that gap so Codex becomes a first-class execution peer — making `task-runner --agent codex` and the fleet orchestrator real, not aspirational.

The three capabilities every engine must offer

Capability	What it does	kimi (exists)	codex today	codex-plugin-cc
Review	Peer-review a diff / branch, read-only	yes	yes (native)	**inherit**
Adversarial	Challenge design, assumptions, trade-offs	yes	yes (native)	**inherit**
Execute (crank)	Build a Task-Spec: write code, run evals, commit, PR	yes	**MISSING**	**ADD ★**

The plugin is purely additive — it never touches OpenAI's existing review/rescue commands.

2 The blueprint it ports (kimi-plugin-cc)

This is **a port, not an invention.** The Kimi plugin already solved every hard problem; we reuse its structure and swap the driver. The load-bearing modules:

Kimi module	Responsibility	Reuse in codex-plugin-cc
broker.mjs	Single dispatcher: subcommands, .env autoload, exit codes	Copy; swap driver import
kimi.mjs	CLI wrapper: JSONL capture, retry, hard+idle watchdogs, bg detach + PID	**REPLACE → codex.mjs**
orchestrate.mjs	buildGraph / topoSort / pathsOverlap — the wave scheduler	Copy verbatim
preflight.mjs	Origin-state check + Task-Spec eval gate	Copy verbatim
git.mjs, state.mjs, context.mjs	diff capture, session meta, CLAUDE.md/rules injection	Copy verbatim
codex-bridge.mjs	Proof Kimi already shells out to codex for verdicts	Reference / delete (native now)

> **Why the watchdog matters (do not simplify it away)**
>
> Kimi's wrapper carries a **hard wall-clock timer** AND an **idle-output watchdog**: a crank that stalls (model looping, hung shell) is SIGTERM'd then SIGKILL'd, and returns terminal exit code 6 — never retried. This is hard-won reliability. The Codex port must preserve both timers.

3 The driver: `codex exec` primitive parity

Grounding was run against the installed CLI (**codex-cli 0.133.0**). Every primitive the Kimi crank relies on has a `codex exec` equivalent — and two are **strictly better**.

Crank needs	Kimi (kimi --print)	Codex (codex exec)	Note
Headless run	--print -p <prompt>	codex exec "<prompt>" (or stdin)	
Working dir	--work-dir <dir>	-C, --cd <DIR>	isolated worktree
Stream events	--output-format stream-json	--json	JSONL to stdout
Final message → file	(parse JSONL tail)	-o, --output-last-message <FILE>	**cleaner**
Write permission	--yolo	-s workspace-write	sandbox policy
Model override	--model	-m, --model	
Resume session	(session dir)	codex exec resume --last	
Structured verdict	(regex VERDICT: X)	--output-schema <FILE>	**real JSON schema**
Extra writable dirs	n/a	--add-dir <DIR>	
Skip git check	n/a	--skip-git-repo-check	worktree edge case

Verified via `codex exec --help`. -o and --output-schema make the Codex port cleaner than Kimi's JSONL scraping.

Reference invocation the crank will emit

```
codex exec \
  --cd "$WORKDIR" \
  --json \
  -o "$SESSDIR/last-message.txt" \
  -s workspace-write \
  -m "$MODEL" \
  "$TASK_PROMPT"        # or: printf '%s' "$TASK_PROMPT" | codex exec -

# resume:      codex exec resume --last --json -o ...
# adversarial: codex review --uncommitted   (native reviewer, no crank spine needed)
```

The crank builds these argv arrays in codex.mjs, mirroring kimi.mjs runOnce().

> **Sandbox choice is a policy decision**
>
> `-s workspace-write` lets the agent edit the working tree but sandboxes shell. For CI/externally-sandboxed runs use `--dangerously-bypass-approvals-and-sandbox`. Default to **workspace-write**; expose an escape hatch via env (`CODEX_SANDBOX=danger-full-access`).

4 File manifest

Structure mirrors kimi-plugin-cc so muscle memory and tooling transfer. New/changed files marked ★.

```
codex-plugin-cc/
  .claude-plugin/plugin.json          # name: codex-crank, marketplace metadata
  commands/
    codex:crank.md            â■■  # execute one Task-Spec
    codex:crank-next.md       â■■  # highest-priority ready task off the DAG
    codex:crank-batch.md      â■■  # dependency-respecting waves
    codex:status.md              # session status (port)
    codex:result.md              # fetch final output (port)
    codex:cancel.md              # cancel bg session (port)
    codex:setup.md               # verify codex CLI, auth, review gate
  agents/
    codex-crank-worker.md     â■■  # subagent that drives one crank (Bash-capable)
  scripts/
    broker.mjs                   # dispatcher (port; swap driver)
    lib/
      codex.mjs               â■■  # codex exec wrapper (replaces kimi.mjs)
      orchestrate.mjs            # wave scheduler (verbatim)
      preflight.mjs              # origin + eval gate (verbatim)
      git.mjs  state.mjs  context.mjs  render.mjs   # (verbatim)
  schemas/
    crank-verdict.schema.json â■■  # for codex --output-schema (review hooks)
  prompts/
    crank-preamble.md         â■■  # context wrapper prepended to task body
  hooks/hooks.json               # optional stop-time review gate
  README.md  CHANGELOG.md  LICENSE (MIT)
```

5 Command contracts

5.1 /codex:crank <task.md>

Execute a single Task-Spec. Write-capable. Returns a session ID. Direct analogue of `/kimi:crank`.

Argument surface

```
/codex:crank <path-to-task.md>
    [--background]               # detach, return session id immediately
    [--model <model>]           # e.g. gpt-5-codex; default from config
    [--resume | --fresh]        # continue latest repo session vs. start clean
    [--auto-commit on|off|on-clean]   # on-clean = commit only if evals pass 1st try
    [--plan-review]             # adversarial plan critique BEFORE dispatch
    [--diff-review]             # adversarial diff review BEFORE commit
    [--force-dispatch]          # override origin-diverged abort
    [--skip-preflight]          # override buggy-evals abort (NOT recommended)
    [--no-context]              # skip CLAUDE.md / rules injection
    [--sandbox <mode>]          # read-only|workspace-write|danger-full-access
```

Process (9 steps, ported)

1. **Validate input** — path exists, matches `tasks/T-*.md`; handle --resume/--fresh.
2. **Pre-flight gates** — origin-state check (abort if touched paths diverged); Task-Spec eval validator (abort if eval bodies are buggy); context injection.
3. **Optional adversarial review** — --plan-review critiques the approach; --diff-review critiques the diff. Backed by `codex review` + --output-schema.
4. **Read task file**; wrap with crank preamble.
5. **Capture pre-run diff** (`broker diff-capture --phase pre`).
6. **Dispatch** via `broker dispatch --mode crank` → codex.mjs → `codex exec`.
7. **Capture post-run diff** (foreground only).
8. **Checkpoint recovery** — cancelled work is stashed to `.codex/state/checkpoints/`; restore on --resume.
9. **Surface results** — foreground: final message + diff delta + session id; background: session id + /codex:status reminder.

5.2 /codex:crank-next

Pick and run the **single highest-priority ready task**. Scans `tasks/` for specs with `status: ready`, selects by priority (P0>P1>P2) whose `depends_on` are all satisfied, dispatches with the same gates. Exits “No ready tasks” if none. Honors --force-dispatch / --skip-preflight / --model.

5.3 /codex:crank-batch <glob>

Run a whole backlog in **dependency-respecting waves** — parallel where safe, sequential where paths overlap. This is where the ported `orchestrate.mjs` earns its keep.

- **buildGraph** topologically sorts by `depends_on`, then assigns waves: a task joins the earliest wave whose deps are already complete AND whose `touches_paths` are disjoint from siblings.
- Doc files (README, CLAUDE.md, CHANGELOG…) are allowlisted — overlap on them does not force serialization.
- `--max-parallel N` caps concurrency (default 4).
- Tasks with overlapping non-doc `touches_paths` **never** run in the same wave — prevents two cranks writing the same file.

```
/codex:crank-batch tasks/T-*.md
/codex:crank-batch tasks/T-*.md --max-parallel 2
/codex:crank-batch tasks/T-*.md --force-dispatch --skip-preflight
```

5.4 Session management (ports)

Command	Purpose
/codex:status [id]	Active + recent jobs for this repo; --wait to block; review-gate status
/codex:result [id]	Stored final output for a finished job (verbatim, unsummarized)
/codex:cancel [id]	SIGTERM then SIGKILL a background job; mark session cancelled
/codex:setup	Verify codex CLI present + authed (codex login); toggle stop-time review gate

6 The shared crank spine

Three engines will eventually share this logic (kimi, codex, glm). Keep it identical across all so a task cranks the same way regardless of driver. The spine is four concerns:

Spine concern	Module	Contract
Wave scheduling	orchestrate.mjs	buildGraph(taskPaths, maxParallel) → {waves, conflicts}
Pre-flight gating	preflight.mjs	preflight(taskPath, repoRoot) → {ok, code, reason}
Eval gate	preflight.mjs	Runs the Task-Spec Exit Check; abort on buggy/failed evals
Session state	state.mjs	session meta: id, status, exit_code, commit_sha, reason

Spine-sharing decision (deferred, flagged here)

If each plugin copy-pastes the spine, it can drift. Options: (a) **vendor-copy** into each repo (simplest, versions independently, risk of drift); (b) publish once as a shared npm dep `@owshq/crank-spine` (single source of truth, adds a release dependency). **Recommendation for v1:** vendor-copy — ship fast, extract to a shared dep only once all three engines exist and the API has stopped moving.

7 Exit codes & the eval gate (R6)

The broker returns a typed exit code on `dispatch`. Ported verbatim from Kimi; this is what makes “eval passes = done” machine-enforced.

Code	Meaning	Behavior
0	ok / dispatched	Session started or completed cleanly
2	origin-diverged	Local branch diverged from origin on touched paths — abort (override: --force-dispatch)
3	buggy-evals	Preflight found broken eval bodies — fix the spec (override: --skip-preflight)
4	review-pause	plan-review / diff-review returned CONCERN|REVISE|REJECT
5	checkpoint-conflict	Resume could not re-apply the stashed checkpoint
6	timeout	Wall-clock or idle watchdog killed a hung crank — terminal, not retried; work left uncommitted

The eval gate is the safety net, not the model’s judgment

Codex is a general coding agent, not one of this repo’s KB-grounded specialists. It gets CLAUDE.md/rules injected as context, but what actually protects invariants (the pytz trap, DECIMAL money, the raw→gold seam) is the **Task-Spec Exit Check**. Keep evals strict; the crank is only “done” when they pass.

8 Phased build plan

Ordered for lowest risk first. Each phase has a machine-checkable acceptance gate.

Phase	Deliverable	Acceptance gate
P0	Scaffold repo + plugin.json + port broker/status/result/cancel/setup	/codex:setup reports codex present + authed
P1	codex.mjs wrapper (exec, --json, -o, watchdogs, retry, bg detach)	Unit: a trivial exec run captures final message + exit 0; a hung run returns 6
P2	/codex:crank single-task (preflight + dispatch + diff capture)	Cranks a real tasks/T-*.md to green; eval gate blocks a task with buggy evals (exit 3)
P3	Port orchestrate.mjs; ship /codex:crank-next + /codex:crank-batch	Batch of 3 tasks with a depends_on edge runs in correct waves; overlapping paths serialize
P4	Adversarial review hooks (--plan-review/--diff-review) via codex review + schema	A REJECT verdict pauses dispatch (exit 4)
P5	Publish to marketplace; wire task-runner --agent codex	task-runner --agent codex dispatches this plugin; fleet-orchestrator lists codex as a worker

Acceptance eval (drop into a Task-Spec Exit Check)

```
# codex-plugin-cc smoke: crank a fixture task to green
set -euo pipefail
node scripts/broker.mjs setup --json | grep -q '"codex":true'
SID=$(node scripts/broker.mjs dispatch --mode crank \
        --prompt "$(cat fixtures/T-hello.md)" --session-id test-1 --json | jq -r .sessionId)
node scripts/broker.mjs result --session-id "$SID" | grep -q 'DONE'
test $(node scripts/broker.mjs status --session-id "$SID" --json | jq -r .exit_code) -eq 0
```

9 Risks & non-goals

Risk / non-goal	Mitigation
Editing OpenAI's official codex plugin in place	DO NOT. Ship as a separate sibling plugin that shells out to the same codex binary; update-safe.
Sandbox too permissive by default	Default -s workspace-write; danger-full-access only via explicit env for CI.
Spine drift across engines	Vendor-copy for v1; extract to shared dep once stable (Â§6).
Codex auth not present in headless/cron	/codex:setup gate + broker preflight surfaces 'codex login' guidance.
Model trusts its own judgment over invariants	Strict Task-Spec evals are the gate, not the model (Â§7).

Non-goals for v1: replacing OpenAI's native review; a GUI; multi-repo cranks; anything the eval gate cannot machine-check.

End of codex-plugin-cc v1 design spec. Companion: **glm-cli-plugin-cc-v1.pdf** (Swimlane 2 — the GLM CLI + its plugin). Both engines converge on one shared crank spine and one eval gate.